5-5 · 파티셔닝과 셔플링 최적화 · 실습 1H

PySpark 배치 스크립트로 데이터 마트 산출

mart.py 한 파일 — 5주차 산출물

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 셸을 exit() 하고 mart.py 를 spark-submit 으로 실행한다

② 두 번 돌려 같은 3행이 나오는 것(멱등)을 확인한다

③ 날짜 인자·로그·종료 코드로 실무형으로 다듬는다

④ mart.py · 마트 조회 결과 · README 실측표를 정리한다

제출 마감 10-30(금) 23:59 — 사이트 「산출물 제출」 페이지에서

실습 흐름

네 단계 — 셸의 --conf 를 코드 안으로

5-3 에서 셸에 길게 주던 접속 설정을 mart.py 안에 담습니다

1. exit · 작성	2. 멱등	3. 실무형	4. 정리
셸 나오기 히어독 mart.py	한 번 더 tail -n 8	인자 · mart.log 종료 코드	README 제출

1단계 — 일부러 실패시키기

셸을 띄운 채 spark-submit 을 내면

셸이 떠 있을 때

WARN Initial job has not accepted any resources

8080 Cores	spark-submit	WARN 반복
4 Total, 4 Used	일을 못 받음	15초마다

VS

셸을 exit() 하면

기다리던 잡이 바로 이어서 돈다

exit()	코어 반납	잡 진행
셸 종료	4개 비움	표가 찍힘

1단계

mart.py 를 히어독으로 — 그리고 제출

```
cd ~/data-eng/week5
cat > mart.py << 'EOF'
# … SparkSession 에 s3a · Iceberg(lake) 설정을 코드로
spark.sparkContext.setLogLevel("WARN")   # 결과 표가 끝에 보이게
# … orders + products → amount → groupBy(order_date)
mart.coalesce(1).writeTo("lake.db.daily_sales_mart").createOrReplace()
EOF
docker compose cp mart.py spark-master:/tmp/mart.py
docker compose exec spark-master /opt/spark/bin/spark-submit \
  --master spark://spark-master:7077 \
  --conf spark.jars.ivy=/tmp/.ivy2 \
  --packages org.apache.hadoop:hadoop-aws:3.3.4,org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:1.5.2 \
  /tmp/mart.py
```

> **히어독 전문은 사이트 본문에서 복사 — 셀을 exit() 한 뒤에 붙여 넣으세요**

1·2단계

첫 실행과 다시 실행 — 같은 3행

첫 실행

version hint WARN 스택 뒤에 표 · data/ 1개

order_date	daily_amount	order_cnt
2026-08-24	113000	2
2026-08-25	1155000	2
2026-08-26	46000	1

VS

다시 실행 · tail -n 8

WARN 없이 같은 표만 · data/ 2개

- createOrReplace — 통째로 교체해 멱등
- 옛 판 파일은 스냅샷으로 남는다
- 정리는 4-4 의 expire_snapshots

3단계

개선판 — 로그는 파일로, 종료 코드는 다음 상자

상자 1 — 실행(exec 가 맨 끝)

```
docker compose cp mart.py spark-master:/tmp/mart.py

docker compose exec spark-master \
  /opt/spark/bin/spark-submit … \
  /tmp/mart.py 2026-08-25 > mart.log 2>&1
```

✓ 상자 2 — 확인

```
echo "종료 코드: $?"
grep "\[mart\]" mart.log
```

> exec 뒤에 함께 붙여 넣은 줄은 찍히기만 하고 실행되지 않습니다 — 상자 하나씩, 프롬프트가 돌아온 뒤에

3단계 — 일부러 실패시키기

인자를 바꿔 세 번 — 종료 코드를 읽는다

인자	종료 코드	[mart] 로그
2026-08-25	0	원천 2행 · 마트 1행 · 완료
2099-01-01	1	원천 0행 · 중단
날짜 없이	0	원천 5행 · 마트 3행 · 완료

날짜를 주면 마트가 그 한 줄로 교체된다 — 날짜 없이 한 번 더 돌려 3행으로 되돌린다

| grep 을 붙인 뒤의 $? 는 grep 의 0 — 그래서 mart.log 로 받고 따로 확인합니다

4단계 — 산출물 정리

5주차 산출물 세 가지

01

① mart.py

배치 스크립트

인자·로그·종료 코드

02

② 마트 조회 결과

mart.log 의 표

또는 콘솔 캡처

03

③ README

실행 줄 그대로

실측표는 본인 수치

> **README 「3. 실행」 = cp → spark-submit … /tmp/mart.py [날짜] > mart.log 2>&1 → echo "종료 코드: $?"**

확인 포인트

이 1시간이 끝날 때 확인할 것

1. 셸을 exit() 한 뒤 spark-submit 이 기다리지 않고 돈다
2. 표 3행 113000·2 / 1155000·2 / 46000·1
3. MinIO warehouse/lake/db/daily_sales_mart/data/ 에 Parquet
4. 종료 코드 0 · 1 · 0 — 마지막에 마트가 다시 3행
5. mart.py · 조회 결과 · README 가 제출 형태로 모였다

5-5 · 실습 1H 마무리

다음은 정리·회고 및 산출물 점검 (0.5H)

5주차를 되짚고 자주 겪는 오류를 정리한 뒤, 클러스터를 stop 하고 6주차 Kafka 를 준비합니다

- ✓ 5주차에 지나온 길
- ✓ 자주 겪는 오류 — Initial job · 4040
- ✓ ls 파일 여섯 → docker compose stop
- ✓ 6-1 Kafka 예고 · week6 폴더

가져갈 것

mart.py · mart.log · README